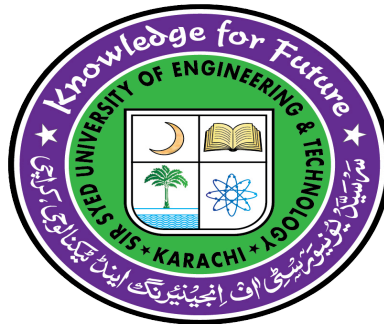


Technical Requirement Document

Group No. 04
Travello AI
BS COMPUTER SCIENCE
BATCH 2022F
Dr. Waleej Haider
Associate Professor
SSUET

Submitted by

2022F-BSCS-156	Huzaifa Rizwan
2022F-BSCS-195	Sameed Uddin
2022F-BSCS-322	Syeda Tooba Fatima



In Partial Fulfillment
Of the Requirements for the degree
Bachelor of Science in Computer Science

Department of Computer Science & Information Technology
Sir Syed University of Engineering & Technology
University Road, Karachi 75300
<http://www.ssuuet.edu.pk>

DECLARATION

In this respect, we hereby declare that this project report titled “**Travello AI**”, which has been presented before the “**Department of Computer Science and Information Technology**”, is an original product developed by us under the supervision of our supervisor “**Dr. Waleej Haider**” and that there has been no plagiarism at all on our part. Moreover, this project work has been prepared to fulfill some of the requirements for the degree of B.Sc. in Computer Science.

Name : Huzaifa Rizwan

Signature: _____

Name : Sameed Uddin

Signature: _____

Name : Syeda Tooba Fatima

Signature: _____

Supervisor:

Signature:

Dr. Waleej Haider
Associate Professor

Date

ABSTRACT

The problem with Pakistan's travel industry is that there is no single digital platform that provides all the services in a cohesive manner. Travelers in Pakistan have to use different platforms in order to plan their flights, book hotels, and make train reservations. There are no services for local travel like those offered by Pakistan Railways, and the problems such as booking a certified airport driver, nearby health centers, and weather alerts are not even considered by any existing platform.

This is where Travello AI makes the difference by providing all travel services through a single mobile application. One can make flight reservations, book hotels, buy train tickets, pre-book an airport driver, search for nearby hospitals, and get weather alerts all from a single app. There is also an AI travel assistant that makes personalized travel recommendations.

The scope of this project encompasses all the core functionalities of booking and assistance features, which can be extended in the future to include more advanced functionalities such as real time services integration and digital payment gateways. Unlike other existing websites such as Booking.com, Skyscanner, Sastaticket, and Bookme, Travello AI is the first website that has incorporated all these features especially for the Pakistani people.

The project will be assessed by its efficiency and its ability to provide satisfactory experience to the users. In the future, the focus will be to integrate more live services from third parties as well as adding more payment options and developing backend infrastructure.

Table of Contents	Page No.
-------------------	----------

Chapter 1: Introduction	5-6
1.1 Background of Study	5
1.2 Project Objectives	5
1.3 Problem Statement	6
1.4 Scope of Study	6
Chapter 2: Literature Review	7-13
2.1 Existing Systems	7-8
2.2 Research Based Literature Review	9-11
2.3 Proposed System	12-13
2.4 Comparative Analysis	13
Chapter 3: Requirement Specifications	14-24
3.1 External Interface Requirements	14-17
3.2 Functional Requirements	17
3.3 Non-Functional Requirements	18-21
3.4 Cost Estimation	22-24
Chapter 4: System Design	24-31
4.1 System Architecture Diagram	24
4.2 High-Level Design: System Operations	25-26

4.3 High-Level Design: System Model	27
4.4 Low-Level Design	28
4.5 Database Design	29
4.6 GUI Design	30
Chapter 5: Business Model	31-32
5.1 Business Model Canvas	31

List of Figures:

Figure 4.1: System Architecture Diagram

Figure 4.2: UML Sequence Diagram - Flight Booking Flow

Figure 4.3: UML Use Case Diagram

Figure 4.4: UML Class Diagram

Figure 4.5: Database Design

Figure 4.6: GUI Wireframes & Design Flow

Chapter 1

Introduction

1.1 Background of Study

The travel industry has expanded dramatically, and these days it's possible to book flights, hotels and transportation all from one app. Mobile-first travel planning is now an expectation globally with smart phones and internet access.

But the domestic travel booking market is not consolidated in Pakistan. Users need to go to several different stations to make bookings for their flights, hotel, and train tickets. Users have to visit multiple dis-connected stations to make bookings for flights, hotel, and train ticket booking separately. The integration of Pakistan Railways, one of the biggest local transport providers, with an existing digital travel solution is not meaningful. Proposed to bridge this gap, Travello AI is a single platform for Pakistani travelers, offering bookings, AI recommendations, and location-based features all in one.

1.2 Project Objectives

Travello AI's key objectives are:

- A unified platform where users can search and book flights, hotels, and Pakistan Railways train tickets all in one place.
- An AI powered travel assistant that gives personalized recommendations based on user preferences and location.
- A verified driver booking feature that allows travelers to pre-book airport transportation and view driver details in advance for safety.
- A healthcare finder that helps travelers locate nearby hospitals and clinics based on their current city or location.
- Weather updates and notification alerts to keep travelers informed about conditions at their destination before and during their trip.
- A clean and simple interface that is easy to use for everyone including people who are not very experienced with technology.
- A scalable system structure that allows future features like real time integrations and digital payment to be added smoothly.

1.3 Problem Statement

Traveling within Pakistan continues to be a complex process, with users having to use several apps to book flights, hotels, or train tickets, and many of these apps don't even feature Pakistan Railways. This leads to confusion, time wasters and makes digital booking a hassle, particularly for those who are less tech-savvy. Current platforms are directed towards international users and do not support local routes, payment methods, and needs. Furthermore, at the airport, travelers encounter safety issues because of the lack of a system that ensures the safety of drivers and pickups. The difficulties mentioned above emphasize the significance of getting a unified travel platform especially created for the travelers of Pakistan which will ease and secure the whole travel journey.

1.4 Scope of Study

The travel industry is booming in Pakistan and there is a need for a reliable local solution that is in digital format. Here in Pakistan, millions of people travel in the country on an annual basis for various reasons including jobs, studies, and holidays, but they don't have one platform which provides them with everything that they need to book. Travello AI addresses this by providing an all-in-one booking experience tailored for the local market. The website features domestic flight, hotel, train booking, airport driver booking, hospital finder and weather alerts. The need for a local travel application is growing rapidly as the number of smartphones and internet connectivity has been increasing in Pakistan. Travello AI can be the perfect option to gain a substantial following, as it is the only single destination for travel solutions specifically for Pakistani travelers.

Chapter 2

Literature Review

This literature review consists of two parts:

(1) Application/Web-based review

(2) Research-based review.

2.1 Existing Systems

2.1.1 Title: Booking.com [1]

Description:

Booking.com is one of the most commonly accessed websites that is dedicated to the online booking of hotels and accommodations.

System Features:

- Comprehensive hotel search and booking
- Reviews and ratings from users
- Flexible booking and cancellation policies
- Comparing prices and offers
- Search based on location

2.1.2 Title: Sastaticket.pk [2]

Description:

Sastaticket is an online ticket booking organization founded in Pakistan allowing customers to search for and book tickets on buses, hotels, and flights. The company also provides unique payment options alongside specialized services for Pakistani customers.

System Features:

- Booking services for buses, hotels, and flights.
- Integrations with local and international airlines

- Comparison of prices from different providers.
- Safe methods of payment online
- A simple booking management system

2.1.3 Title: Skyscanner [3]

Description:

Skyscanner is an international online travel search engine that allows people to make comparisons between flights, hotels and cars. The service is popular due to its flexible search options and possibility to track prices.

System Features:

- Round-trip and multi-city search
- System for price alerts
- Hotel and transport search combined
- Filtering by different criteria

2.1.4 Title: Bookme.pk [4]

Description:

Bookme.pk is the top online e-ticketing and travel booking portal in Pakistan where users can book flights, trains, buses, hotels, cars, tours, and other travel related services via just one application. This portal provides its users with online payment facilities, instant issuance of e-tickets, and booking management and thus is considered one of the best travel booking portals in Pakistan.

System Features:

- Integrated booking of flights, trains, buses, and hotels
- Online booking management with instant e-tickets
- Various secure modes of payment
- Domestic and international flights booking
- Car rental and other travel services

2.2 Research Based Literature Review

Description:

The following research paper discusses the development of an AI-powered travel recommendation system that makes use of machine learning and personalized recommendations to assist users in planning their travels. The travel recommendation system integrates various travel services such as weather forecast, maps, destination recommendations, and trip planning. The research primarily concentrates on minimizing the complexity associated with shifting from one travel application to another. [5]

Main Features:

- AI-powered personalized travel recommendations
- Budget and duration-based suggestions
- Weather and location integration
- Dynamic itinerary generation
- Google Maps integration
- Context-aware recommendations
- Multi-API integrations for travel planning
- Estimating cost of travel and stay

Limitations:

- Recommendation-focused rather than a booking platform
- Not designed for Pakistani passengers
- No Pakistan Railways interface
- Not an airport cab driver verification tool
- No nearby hospital or medical facility search
- Does not cater to domestic travel

Reason Why Travello AI is a Superior System:

The Travello AI system not only suggests recommendations via AI technology but also has an entire booking system for flights, hotels, and even booking for the train services of Pakistan Railways in one single mobile application. In contrast to the proposed research-based system, Travello AI is designed especially for Pakistani users and contains features like verification of airport drivers booking, hospital finder nearby, weather alert, and travel recommendations based on local criteria.

Description:

Pocket Safar is an artificial intelligence based smart recommendation system intended for improving personalized recommendations and budgeting for travelers. The aim of the study is to combine functions of itinerary generation, recommendations, emergency aid, and collaborative planning in a single system. [6]

Main Features:

- Artificial intelligence driven travel recommendation system
- Dynamic itinerary planning
- Budget optimization algorithms
- Booking flights, hotels, trains, cabs, cruises
- Community-based travel collaboration
- Geolocation analytics
- Emergency essential detection
- Hybrid recommendation techniques

Limitations:

- The system caters to general international tourists.
- It does not cater to domestic travel issues in Pakistan.
- There are no facilities for payments like travel systems of Pakistan.
- The system does not offer validated transport from airports.
- There is no facility for hospital finders.
- There is no integration with Pakistan Railways.

Why Travello AI Is Superior:

Travello AI concentrates on resolving travel-related issues of Pakistani travelers. Even though Pocket Safar is centered on itinerary-making and algorithmic recommendations, Travello AI unites all travel facilities in one place. Travello AI provides the service of booking in Pakistan Railways, airport driver verification for safe transportation, healthcare search facility, weather alerts, and travel recommendation facility. Moreover, Travello AI is designed to make travel easier for users who have little knowledge about technological devices.

Description:

This research paper presents an explainable AI-based travel recommendation system that incorporates machine learning algorithms and SHAP analysis to offer a transparent and personalized travel recommendation. The central theme of the study is enhancing the trustworthiness of AI-based travel recommendations. [7]

Main Features:

- Explainable AI (XAI)-based recommendation system
- Destination recommendation with personalization
- Gradient boosting machine learning algorithm
- SHAP analysis for transparency of recommendation
- User behavior analysis
- Intelligent recommendation engine
- Data-driven travel planning

Limitations:

- Primarily deals with recommendation transparency only
- Lacks complete travel booking services
- No train booking integration
- Does not have airport driver verification facility
- No healthcare support feature
- Not suitable for Pakistani domestic travel needs

Why Travello AI is a Better Solution:

As compared to ExplainableTrip, Travello AI offers much more than just AI recommendations. It provides a complete travel ecosystem that can serve the needs of Pakistanis. In addition to AI assistance, Travello AI also provides booking services for flight tickets, hotel bookings, train tickets, and other travel needs. It has features such as airport driver verification booking, weather notifications, healthcare facility search, and much more.

2.3 Proposed System:

Travello AI is a travel app that's mobile oriented which allows a user to book flights, reserve hotels, pre-book and post book verified airport drivers, find nearby hospitals and clinics and get weather updates and alerts from one interface. The AI travel assistant also offers personalized suggestions based on the user's destination and preferences.

All the above needs are not catered by existing platforms such as Booking.com, Skyscanner, and Sastaticket, none of whom support Pakistan Railways, verified transportation, healthcare finding, or weather alerts. MakeMyTrip is the closest, but is

tailored to the Indian market and does not cater to the needs of Pakistani travellers. Travello AI addresses each of these gaps in a single solution.

2.4 Comparative Analysis:

Feature	Booking.com	Sastaticket .pk	Skyscanner	Bookme.pk	Travello AI
Flight Booking	✓	✓	✓	✓	✓
Train Booking	✗	✗	✗	✓	✓
Hotel Booking	✓	✓	✓	✓	✓
Unified Platform	✗	✗	✗	✓	✓
Price Comparison & Recommendations	✓	✓	✓	✗	✓
Flexible Search & Filtering	✗	✓	✓	✓	✓
Local Service Integration	✗	✓	✗	✓	✓
Travel Bundle Creation	✗	✗	✗	✗	✓
Airport Driver Booking	✗	✗	✗	✗	✓
Hospital Finder	✗	✗	✗	✗	✓
Weather Alerts	✗	✗	✗	✗	✓
AI Travel Assistant	✗	✗	✗	✗	✓

Above is a comparison of all features that are present in the available platforms and none of them offer all the features that are needed by the modern travellers in Pakistan. **Travello AI** is the only platform that offers everything from flight booking, train ticketing, hotel reservations to verified airport transportation, hospital search, weather forecasts and an AI travel assistant all in one. **Bookme.pk** provides travel booking services for flights, trains, buses, and hotel packages specific to Pakistan, but it lacks AI-powered travel planning, intelligent recommendations, health support, and weather forecasting services. The comparison brings up an important point: the lack of a comprehensive, intelligent travel ecosystem in Pakistan is an opportunity that Travello AI aims to fill.

Chapter 3

Requirement Specifications

3.1 External Interface Requirements

3.1.1 User Interfaces

It includes a user interface which is mobile-based and which allows the user to interface with the program in an efficient manner. It is supposed to accommodate all the functionalities of the program, including registration, login, searching for travel information, booking, and profile management.

Users interact with the program through a series of screens where they can do the following:

1. Register themselves in the application.
2. Perform searching, comparing and selecting between flights, trains, and hotels.
3. Provide information on passengers and make bookings.
4. Get weather information, alerts about nearby hospitals, and recommendations for travel from artificial intelligence.

The user interface will be easy to use, consistent, and accessible to both new and returning users through efficient navigation and interaction processes.

3.1.2 Hardware Interfaces

This system interfaces with typical hardware components that are required for proper functioning of the applications and which operate on mobile devices. Some of them include:

- Processor: ARM or x86 based processor, with speed of 1.0 GHz or higher.
- RAM: Minimum 1GB RAM is needed. 2 GB is preferable for smooth operation.
- Display Screen: This display screen shows the UI of the system.
- Touchscreen: This component enables users to interact by touching and scrolling.
- Device Storage: Minimum 100 MB free storage is required for installation of application.
- Network Interface: It is required for network connections.
- Camera/Gallery: For uploading a profile picture.

3.1.3 Software Interfaces

The Travello AI system integrates with the following software components, libraries, and external services:

1. Mobile Application Framework:

The client application is developed using Flutter (Dart SDK 3.x), a cross-platform mobile development framework. It runs natively on Android devices and communicates with the backend exclusively via HTTPS REST API calls.

2. Backend Framework:

The server-side application is built with FastAPI (Python 3.11), deployed on Render.com. It handles all business logic, routing, and orchestration of external API calls. It exposes a RESTful API consumed by the Flutter client.

3. Database and Authentication :

Supabase with PostgreSQL 15 is used as the main database. It stores user profiles, bookings, payments, passengers, notifications, reviews, and support messages. Supabase Auth handles registration, login, session management, and JWT token issuance. Supabase Storage stores user profile images.

4. Flight Data:

Flight results are generated from seeded mock data covering all major Pakistan domestic routes. The AviationStack API supplements this with real flight schedules when available. The API returns flight data including airline name, departure and arrival times, and flight numbers. Responses are cached for 24 hours to stay within free tier limits.

5. Hotel Search :

Hotel availability is retrieved via the TripAdvisor endpoint on RapidAPI. HTTP GET requests are sent with city name, check-in/check-out dates, and guest count. If RapidAPI does not return enough results, Google Places API supplements with up to 20 additional hotel listings nearby. OpenStreetMap is used for city geocoding. Results from all sources are merged and de-duplicated. Mock hotel data is used as a fallback when both APIs are unavailable.

The Travello AI system is integrated with the following software, libraries and external services:

1. Mobile Application Framework:

The client application is created in the Flutter (Dart SDK 3.x) mobile application development platform. It is a native app on Android devices and communicates with the backend only through HTTPS REST API calls.

2. Backend Framework:

The server-side application is created using FastAPI (python 3.11) and deployed onRender.com. It takes care of all business logic, routing and orchestration of calls to external APIs. It exposes a RESTful API that is consumed by a Flutter client.

3. Database and Authentication :

The primary database is Supabase that has PostgreSQL 15. It manages user profiles, booking, payments, passengers, notifications, reviews, and support messages. Supabase Auth is responsible for user registration, user logins, session management, and creating JWT tokens. User profile images are stored in Supabase Storage.

4. Flight Data :

The results of the flights are obtained from mock seeded data for all major domestic routes of Pakistan. The AviationStack API provides this and more when there are real flight schedules. The API provides the data for flights, such as their numbers, names of airlines, and departure and arrival times. Responses are cached for 24 hours to ensure that they stay within the free tier.

5. Hotel Search :

The data for the hotel availability is provided by the Tripadvisor endpoint on RapidAPI. City name, check in and check out dates and guest count are sent as an HTTP GET request. Google Places API will provide up to 20 additional hotel listings in the vicinity, if they are not provided by RapidAPI. City geocoding is done using Nominatim (OpenStreetMap). All results from all sources are combined and deduplicated. If both APIs are not available, they fall back on mock hotel data. .

6. Location and Destination Data :

Google Places API is used for destination search auto-complete and for finding nearby hospitals, pharmacies, and clinics. Requests include coordinates and place type and responses return place names, addresses, and location data.

7. Weather Data:

Current weather conditions for Pakistani cities are fetched from the Google Weather API as the primary source. Open-Meteo API serves as a fallback when the Google API is unavailable. Both APIs accept city coordinates and return JSON data including temperature, conditions, and humidity. Weather responses are cached per city for 5 minutes and across all cities for 2 hours to reduce API calls.

8. Email Service - Gmail SMTP:

Transaction emails are sent through Gmail SMTP utilizing smtplib library within python through port 587 with STARTTLS encryption. Email categories include booking confirmation emails, OTP verifications, and administrative support emails.

3.1.4 Communications Interfaces

1. HTTPS REST API:

The Flutter application communicates with the FastAPI backend over HTTPS running on port 443. The backend server is hosted on Render.com with automatic TLS termination. Requests and responses use JSON data format with UTF-8 encoding. Common HTTP methods include GET to fetch data, POST to create a resource, PUT to modify resources, and DELETE to delete resources.

2. JWT-Based Authentication:

Once the user logs in, Supabase Auth generates a signed JWT token. In the Flutter application, the token is extracted from the Supabase session and appended as the Bearer Token in all authenticated requests. The FastAPI backend validates the token signature using the Supabase JWT secret before handling the request. The token expires after an hour and is automatically renewed using the Supabase Flutter SDK.

3. SMTP - Email Communication:

SMTP on port 587 and STARTTLS encryption through Gmail SMTP is used to send emails from the backend. App passwords are used instead of direct credentials. Emails include booking confirmation email sent after payment and support notification email sent upon the submission of a support message.

4. TLS/SSL Encryption:

TLS 1.2 or 1.3 protocol is employed to ensure that all the data will be transmitted securely from the client to the backend and to third-party APIs. No sensitive information will be transferred through HTTP. Supabase database connections are safe on port 5432. Supabase database connections are safe on port 5432.

5. Data Format and Standards:

All requests and responses of APIs contain the body that will be transmitted in JSON. Date will be presented by ISO 8601 date format (YYYY-MM-DD). Monies will be denoted with floating-point number in PKR. Primary keys in all databases will have the form of UUIDs.

6. Communication Security:

AviationStack, RapidAPI, Google Places and Gmail SMTP API keys are saved as environment variables on the server and not available to the client. In the same way, Supabase Service Role Key for admin operations will be saved on the server side. Row Level Security (RLS) policies in Supabase database make sure that users have access only to their data.

3.2 Functional Requirements

ID	Requirement
REQ-1	User Authentication and Management: User account registration, login, profile management, and password recovery shall be performed through email and password verification, login with the Remember Me feature, and email OTP for password recovery, respectively. Profiles may be edited or deleted with all related data from the system. Session information is maintained until the session is closed by logging out of the system.
REQ-2	The Multi-Service Booking System: The system should provide options for booking flights, trains, and hotels. Services can be searched based on location and date, filtering used, seats/rooms chosen, details viewed, and booking confirmed. The passenger details should be entered through validation.
REQ-3	Payment Handling & Booking Handling: The system will facilitate credit/debit card payment transactions using Luhn verification, CVV, and expiry checking. Booking confirmations are sent by email. Customers have access to view booking history with status tracking, booking cancellation, and rating/comment submission.
REQ-4	Search & Filtering Mechanism: The system shall offer smart search along with auto-complete options for cities, airports, and train stations. Users can further filter their searches based on cost, time, number of stops, and preferred operators with sorting choices like cheapest, fastest, and recommended. Featured offers shall be personalized based on the user's home city.
REQ-5	User Support & More Features: The system shall offer support through messaging with tracking of queries and responses from the admin. Users will receive in-app notifications about their bookings and updates. More features include home city personalization, health service search with maps and directions, real-time weather for cities in Pakistan, and onboarding screens for new users.

REQ-6	AI Travel Assistant: The system shall provide a conversational assistant that interprets travel requests in natural language and searches flights, trains, hotels and rides on the user's behalf. The assistant requests any missing trip details, presents a priced summary, and creates a booking only after the user explicitly confirms it. Card details and identity documents are never accepted through the chat and are collected only through the standard secure forms. Conversation history is stored so a user can resume an earlier chat.
-------	---

3.3 Other Nonfunctional Requirements

3.3.1 Performance Requirements

ID	Performance Requirement	Details
PERF-1	Response Time	The application should load the home page within 2 seconds using a regular 4G network. The flight, train, and hotel search results should be available within 3 seconds of initiating the search query.
PERF-2	Smooth-UI Animations	All UI transitions and animations need to be performed at a minimum of 60 FPS with Flutter animation library with duration from 200 to 400 ms.
PERF-3	Data Persistence	The user authentication state and the local preferences such as home city and remember-me flag are stored in SharedPreferences to retrieve them in less than 100 ms. Booking history and the user data are stored in Supabase cloud database and are loaded on demand. The session state is persisted across app restarts without the need of user re-login unless the user logged out.
PERF-4	Memory Management	The application effectively handles the memory usage through proper disposing of controllers, lazy loading of list view with shimmer effect while data loading, and maintaining cached data not exceeding 50 MB.
PERF-5	Offline Capability	Network failures are gracefully managed with user-friendly error messages and retry options. Pull-to-refresh is present for the home, weather, and healthcare screen to refresh data on connection. The weather data is provided by the backend cache (5-minute TTL for each city; 2 hours for all cities).

3.3.2 Safety Requirements

ID	Safety Requirement	Details
SAFE-1	Protection Against Cyber Threats	Sanitization is performed for the prevention of SQL injection attacks and XSS attacks in the application. User inputs are validated via RegExp patterns. Supabase Row Level Security policies define boundaries for data access within the database.
SAFE-2	Data Backup & Recovery	User data and booking history are stored in Supabase cloud Postgres database persistently. Local state including city preferences and session state are kept in SharedPreferences. Exceptions and try-catch blocks are used for maintaining data consistency and preventing crashes due to data errors..
SAFE-3	Secure Payment Processing	Card numbers are validated via the Luhn algorithm. The CVV number and expiration date are validated for format requirements. Masked input for password is applied.
SAFE-4	Session Management	User sessions are handled using Supabase JWT tokens with auto refresh. Sensitive local storage data is cleaned on user logouts. User deletion results in complete removal of all user data in the database including bookings, reviews, preferences, and payment information.
SAFE-5	Unauthorized Booking Protection	Bookings can never be updated without authorization. Booking has a unique booking id. Booking and payment endpoints need to have an authenticated user session token. The support reply endpoint for admin needs a secret key from the server side.

3.3.3 Security Requirements

ID	Security Requirement	Details
SEC-1	Password Security & Storage	The password has to be at least 8 characters long with password strength validation for upper case letters, lower case letters, number, and special characters. The passwords are never saved locally. Passwords are encrypted using Supabase Auth.
SEC-2	Authentication & Authorization	There are two kinds of user roles: Registered User role, which has all permissions for booking, payment, profile, and history; and Logged-out User role, which is limited and is always redirected to either logging in or registering.
SEC-3	Sensitive Data Encryption	There is no storing of payment card data. The personal data is accessed via AuthService that uses Supabase. API keys and secrets are stored in server-side environment variables and never hardcoded in the code.
SEC-4	Brute Force Protection	Generic login error messages prevent user enumeration. Supabase Auth enforces rate limiting on authentication attempts.
SEC-5	Secure Communication	All API communication uses HTTPS with TLS 1.2+. Supabase JWT tokens authenticate every backend request. Sensitive configuration values are stored in server-side environment variables.

3.3.4 Software Quality Attributes

Attribute	Specification	Implementation Details
Adaptability	Supports different screen sizes and orientations.	Implemented using LayoutBuilder, MediaQuery, and responsive utility helpers.
Availability	High availability with cloud-backed services.	Supabase cloud backend with graceful error handling and retry mechanisms.
Correctness	Strict validation for email, phone, password, card number, CVV, expiry, name, and CNIC.	Implemented via DSValidators class with real-time validation and Luhn algorithm for card numbers.
Flexibility	Supports addition of new services and payment methods.	Modular architecture using GetX state management and reusable components.
Interoperability	Integrates with external APIs and device features.	AviationStack API (for flights), TripAdvisor using RapidAPI (for hotels), Google Weather API with fallback to Open-Meteo (for weather), Google Places API (for healthcare), Supabase (for authentication and database), Gmail SMTP (for email).
Maintainability	Clean architecture with organised folders and reusable code.	Separation into screens, services, controllers, models, and utils.
Portability	Runs on Android.	Built with Flutter SDK ensuring cross-platform support.
Reliability	Handles errors gracefully without crashes.	Try-catch blocks, null-safety, validation, and loading states throughout.
Robustness	Handles invalid inputs and edge cases.	Input formatters, null checks, boundary validations, and fallback UI states.
Testability	Modular design supports independent testing.	Validators and services are structured for unit-test compatibility.
Usability	Provides intuitive and modern UI/UX.	Material Design 3, smooth animations, pull-to-refresh, and shimmer loading effects.

3.4 Cost Estimation

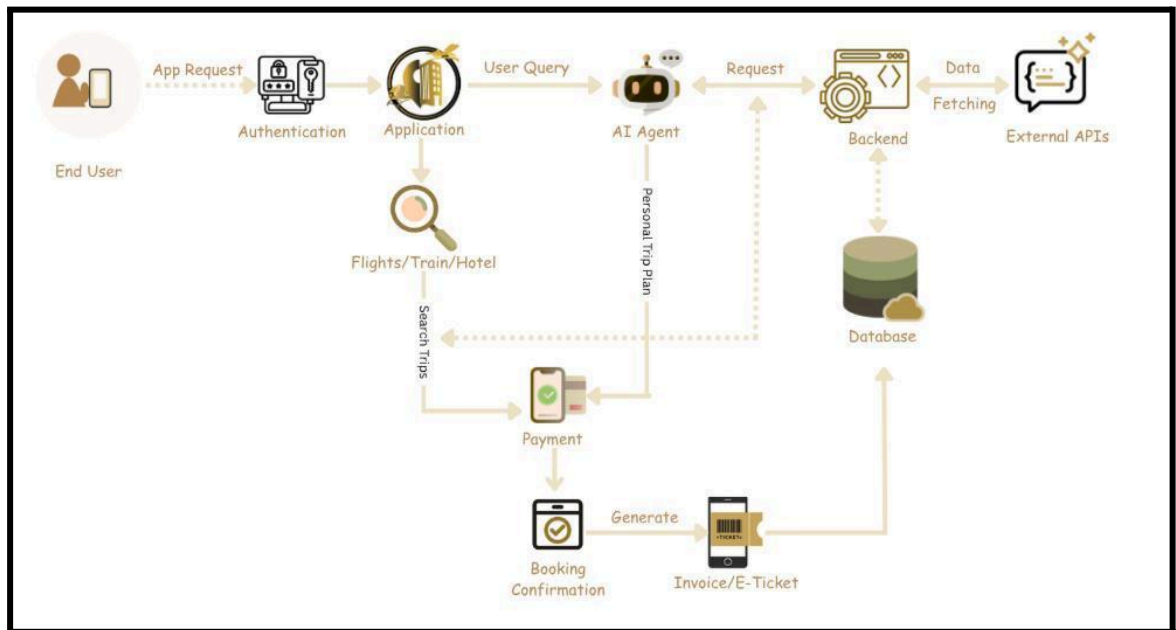
S.No	Project Expenditure	Details	Cost
1	Software Tools and APIs	Google Weather API (Free tier - \$200/month credit covers usage)	Free Tier
		Google Places API (Free tier - \$200/month credit covers usage)	Free Tier
		AviationStack API (Free tier - 100 requests/month)	Free Tier
		TripAdvisor Hotels via RapidAPI (Free tier)	Free Tier
		Open-Meteo Weather API (Fully free, no key required)	Fully Free
		Supabase (Auth, Database, Storage – Free tier)	Free Tier
		Gmail SMTP (Free via App Password)	Free Tier
	Software	dbdiagram.io (Free) – Database schema design	Free
		Adobe Photoshop (Free tier) - Standee, Brochure, Poster Design	Free Tier
		Canva - Used for System Architecture and Flow Diagram	Free
	Frameworks	Flutter SDK (Open Source)	Open Source
		FastAPI Framework (Open Source)	Open Source
		Python 3.11 (Open Source)	Open Source
	Software Tools	VS Code	Free

		• Android Studio	Free
		• GitHub (Free tier)	Free Tier
		• Postman API Testing (Free tier)	Free Tier
	Servers	• Render Cloud (Free tier) – Backend hosting	Free Tier
		• Supabase Cloud (Free tier) – Database & Auth server	Free Tier
	Sub Total		Rs. 0
2	Hardware Cost	• Personal Laptop used for development (Pre-owned)	
		• Android Device for testing (Pre-owned)	
	Sub Total		Rs. 0
3	Networking Cost	• Internet connection (Existing home subscription)	Rs. 0
	Sub Total		Rs. 0
4	Domain Name and Hosting Cost	• Domain Name: Using Render free subdomain	Rs. 0
		• Backend Hosting: Render Free Tier (Authorization 1\$ Fee)	Rs. 1\$
		• Database Hosting: Supabase Free Tier (500MB included)	Rs. 0
	Sub Total		Rs. 1\$
	Grand Total		Rs. 1\$

Chapter 4

System Design

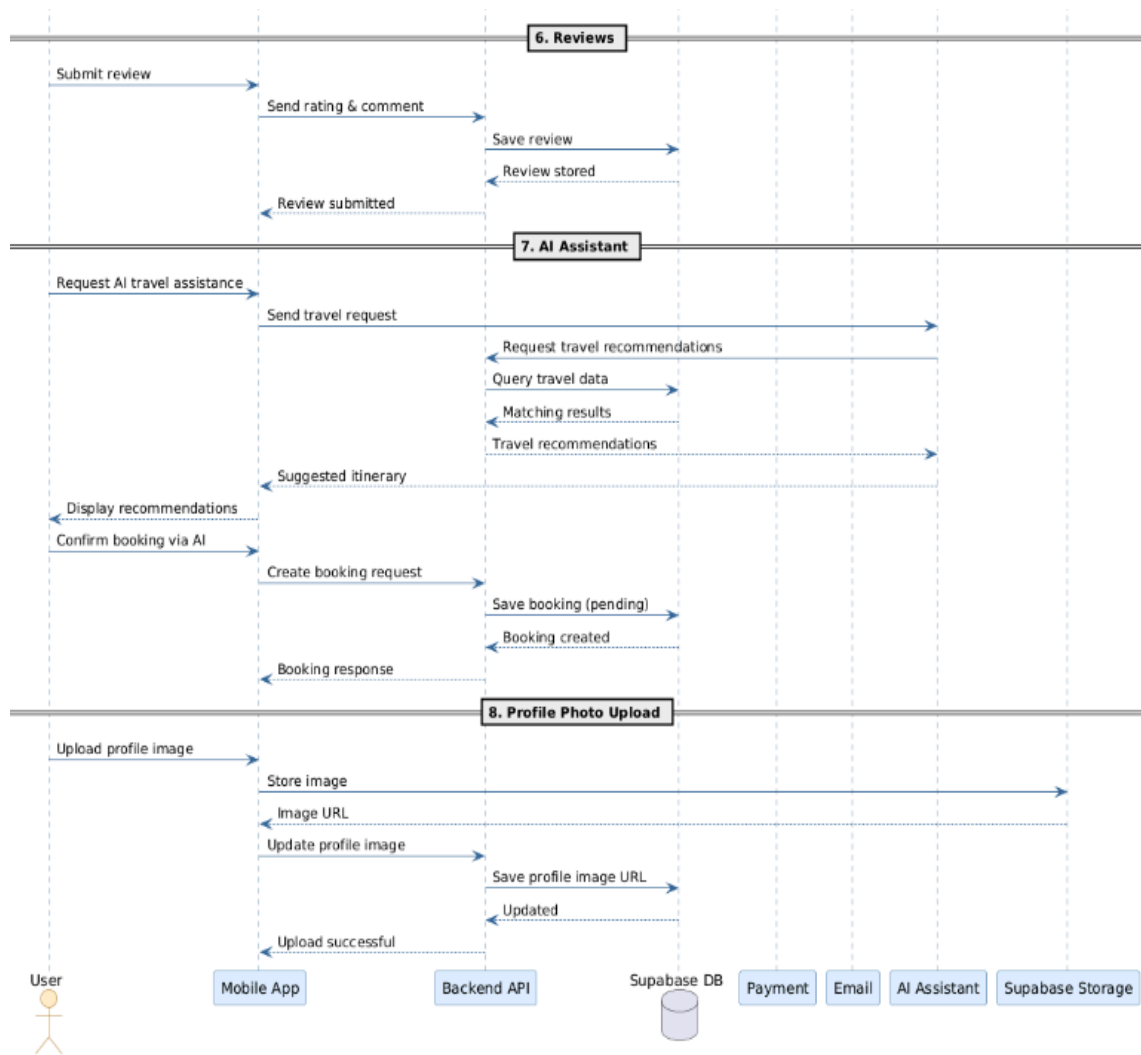
4.1 System Architecture Diagram



Once the end-user starts the application, he gets authenticated in order to access any functionality of the application. Upon logging in, the end-user is able to search flights, trains, and hotels on his own or ask the AI Agent to give him personalized travel suggestions. The AI Agent makes API calls to the backend which retrieves real-time information from various external APIs such as flights, hotels, trains, weather and health services. All user information, bookings and transactions are stored in the database. Upon choosing the service and making payment, the system creates a booking confirmation. Further, an invoice and ticket are created and sent to the user.

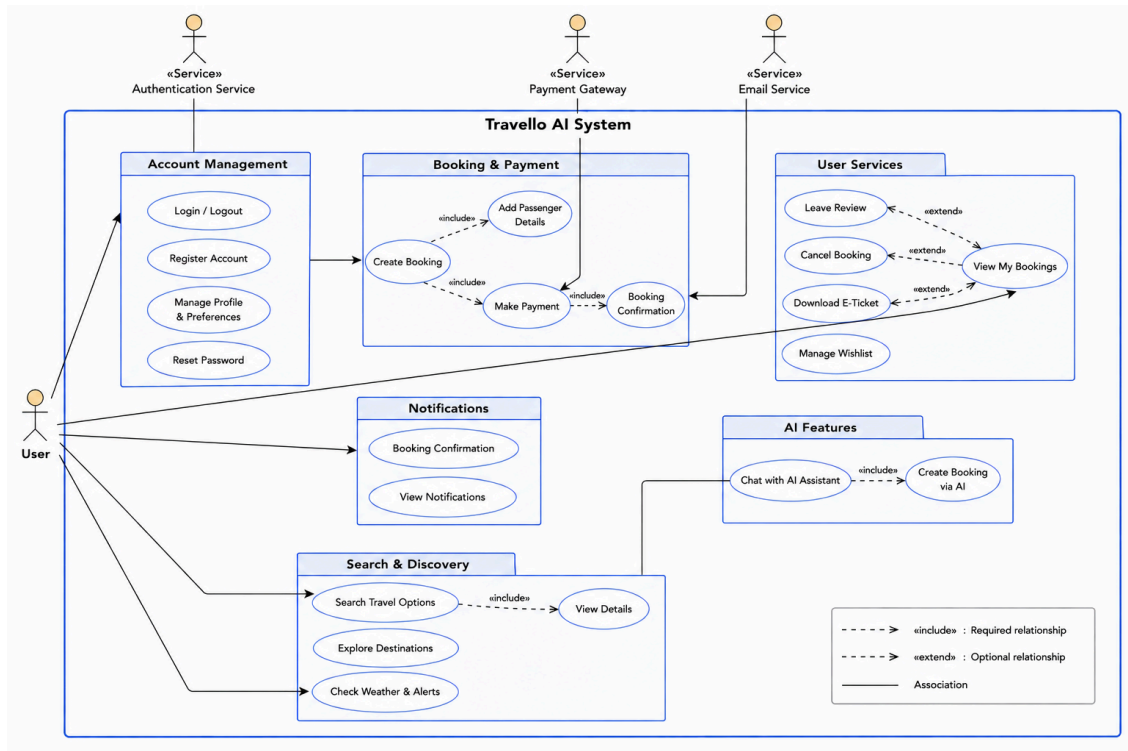
4.2 High Level Design: System Operations (SEQUENCE DIAGRAM)





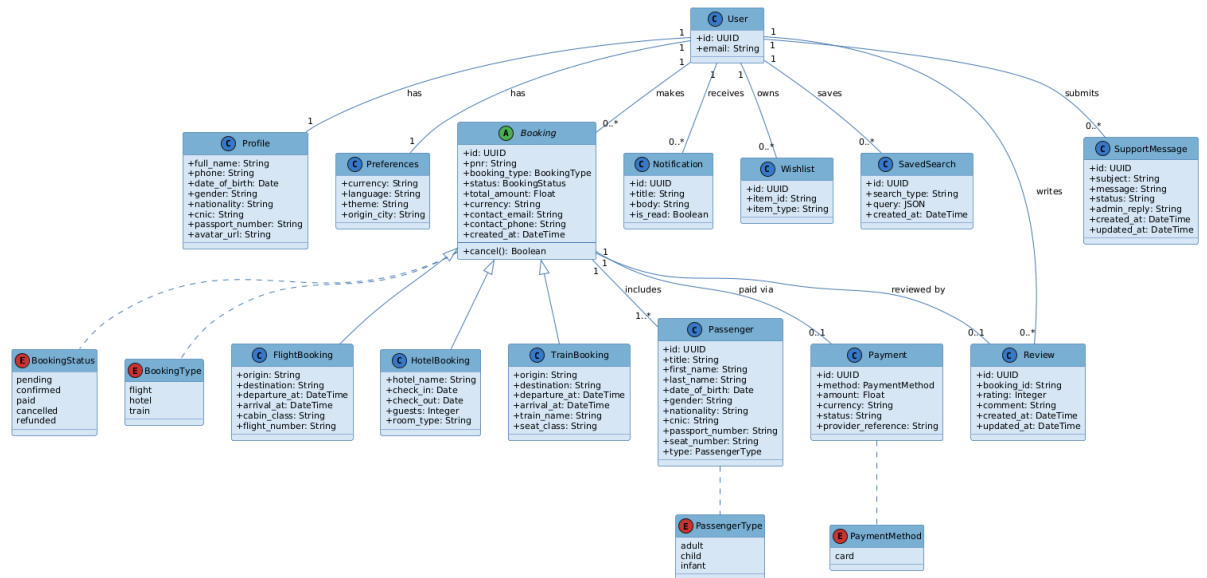
The diagram illustrates the flow of all major system interactions on eight different tabs: Authentication, Search and Plan, Booking and Passengers, Payment and Confirm, My Bookings, Reviews, AI Assistant, and Profile Picture Upload. The process of authentication involves user login and verification by the backend and issuing of the JWT access token. Search and planning involve requests by the app for flights, hotels, or trains from the backend, which will query external APIs and return cached results to the user. In the booking flow, the user chooses an option and enters passengers' information, and the backend stores that as a booking request in the database with the status pending. The process of payment involves validation of the card, changing the booking status to confirmed, storing the payment in the database, and sending a confirmation email to the user. Other flows include booking cancellation, review submission, AI assistant trip recommendations, and uploading of the profile picture to Supabase storage.

4.3 High Level Design: System Model (UML USECASE DIAGRAM)



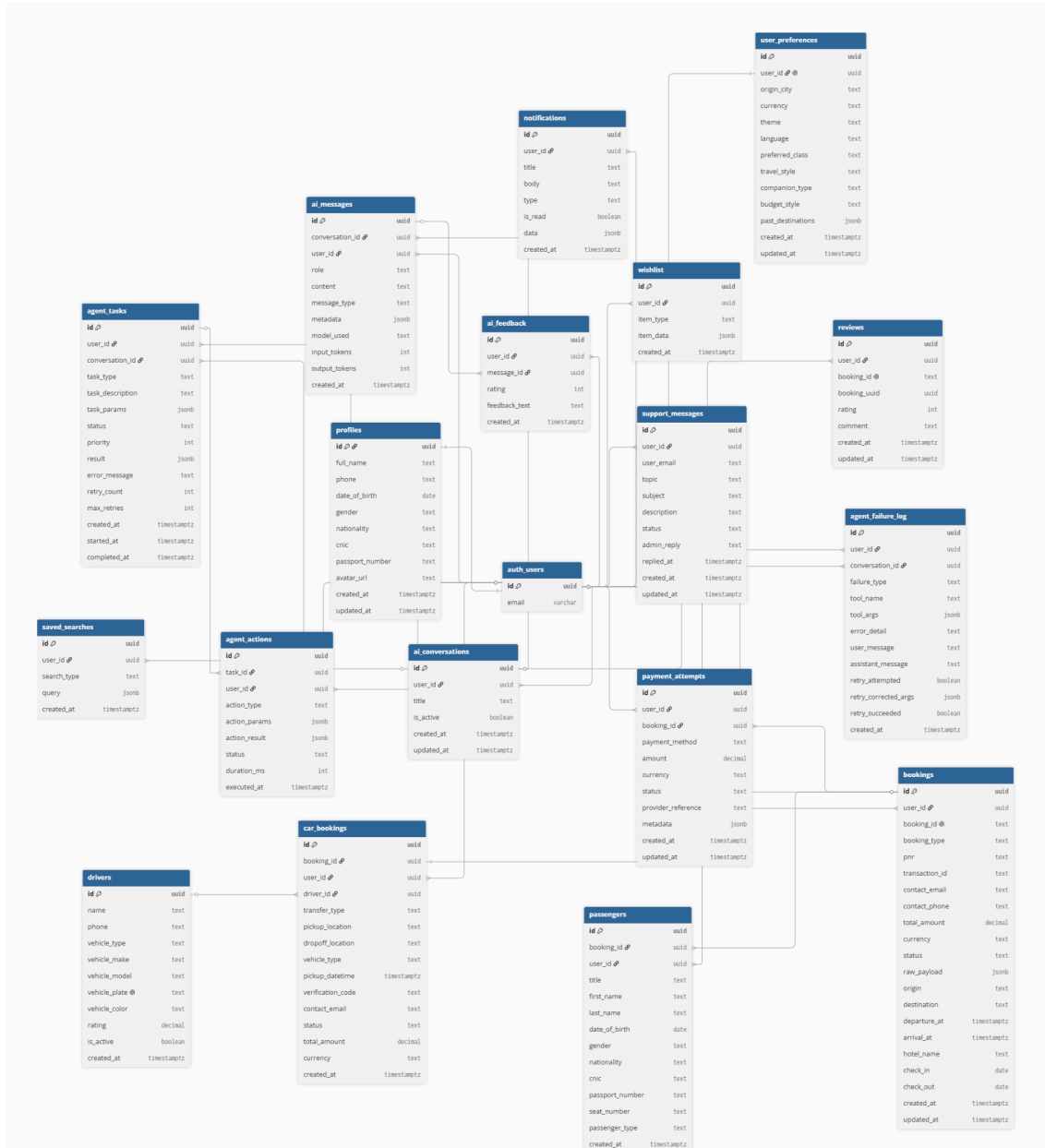
As shown in the diagram below, all significant user interaction processes with the Travello AI system happen within six modules: Account Management, Booking and Payment, User Services, Notifications, Search and Discovery, and AI Features. The first module is responsible for user account-related activities such as registration, logging into an account, profile and password management, while the second one enables users to create a booking by entering passenger details and making a payment using the Payment Gateway. Furthermore, Search and Discovery helps users find the right travel options, explore destination cities and receive weather warnings, whereas User Services offer the opportunity to view and cancel bookings, download electronic tickets, leave reviews, and manage the wishlist. Notifications provide users with updates regarding their booking statuses. The AI Features module enables users to communicate with the AI Assistant, which automatically creates bookings on users' behalf.

4.4 Low Level Design (UML CLASS DIAGRAM)



The following diagram illustrates the data structure of Travello AI where the User object is at the center and is associated with Profile, Preferences, Booking, Notification, Wishlist, SavedSearch, and SupportMessage objects. Booking is the main object which is further related to three specific Booking objects such as FlightBooking, HotelBooking, and TrainBooking with data such as origin, destination, dates, and cabin type. In each booking there will be a Passenger object having attributes such as name, DOB, Nationality, CNIC, Passport Number, and is related to the Payment object having attributes such as Amount, Status, and Provider reference and the only allowed payment option is card. Other objects are as follows.

4.5 Database Design (PHYSICAL DATABASE DIAGRAM)



The diagram illustrates the user flow and interface for a travel application. It includes a flowchart at the top and detailed wireframes for various screens below.

Flowchart:

- User** (Start)
- Passenger Details** (After clicking the Search Flight button, five pages Passenger, Facilities, Checkout, Payment, and Done are displayed sequentially. The user enters details and selects options according to their preferences. The same flow will be followed for train and hotel bookings.)
- Flight Booking** (Includes: Select departure city, Select arrival city, Select date, Search Flights)
- Weather Updates** (Includes: Weather Updates, Active Weather Alerts)
- Healthcare Guidance** (Includes: Emergency Helpline, Nearby Hospitals)
- Travelo AI** (Includes: Travelo AI, Smart AI Travel Booking Chatbot)
- Explore Pakistan** (Includes: Explore the most beautiful places around...)
- My Bookings** (Includes: My Bookings, My Tickets, Booking History, Saved Packages, Saved Searches)
- FAQ's** (Includes: How do I create an account on Travelo AI?, How do I book a flight?, Can I book hotels on Travelo AI?, Can I book trains and hotels as well as flights?, How do I cancel my booking?, What payment methods are supported?)
- Showing nearby hospitals** (Based on the selected city)

Wireframes and Annotations:

- Featured Packages Shown On Home Page:** Shows a grid of featured packages with images and details.
- Showing Notifications:** A notification screen with a bell icon and a message: "You're all caught up! No new notifications on the moment."
- List of All Packages for Flight Booking:** A list of flight packages with details like destination, price, and availability.
- Passenger Details:** A screen for entering passenger information, including name, age, gender, and contact details.
- Flight Booking:** A screen for selecting flight details, including departure city, arrival city, and date.
- Weather Updates:** A screen showing current weather conditions and forecasts.
- Healthcare Guidance:** A screen providing emergency helpline numbers and nearby hospitals.
- Travelo AI:** A screen featuring a Smart AI Travel Booking Chatbot for user assistance.
- Explore Pakistan:** A screen showcasing beautiful places around Pakistan.
- My Bookings:** A screen displaying a user's booking history and saved packages.
- FAQ's:** A screen with frequently asked questions and answers.
- Showing nearby hospitals:** A screen showing a map of nearby hospitals based on the selected city.

Chapter 5

Business Model

5.1 Business Model Canvas

The following Business Model Canvas illustrates the main elements that make up the business model of how Travello AI creates, delivers, and captures value for Pakistani travelers.

5.1.1 Key Partners:

- AviationStack – provides domestic flight schedules.
- RapidAPI (TripAdvisor) – provides hotel offers.
- Pakistan Railways – provider of train routes and schedules.
- Google – places API to search for healthcare facilities, and weather API to get notifications.
- Supabase – provides databases, authentication, and files hosting service.
- Render.com – backend server hosting.
- Airport drivers, verified.

5.1.2 Key Activities:

- Development and maintenance of the Flutter mobile app.
- Integrations and management of the third-party APIs for flights, hotels, trains, weather, healthcare.
- Processing bookings, payments, and sending of confirmation emails.
- Operation of AI travel assistant.
- Onboarding of airport drivers to the platform.
- Support handling and admin responses.

5.1.3 Key Resources:

- Flutter mobile application for Android.
- Backend FastAPI deployed on Render.com.
- Supabase database that stores all user, bookings, and payments-related data.
- Integrations of third-party APIs for travel data.
- AI travel assistant based on the LLM.
- Transactional email service – Gmail SMTP.

5.1.4 Value Propositions:

- Unified platform to book flights, hotels, and Pakistan Railways services.
- The only travel application that caters specifically to Pakistani domestic travelers.
- Authenticated airport driver bookings to ensure safe pick-up.
- AI-enabled travel guide offering personalized recommendations.
- Transparent pricing with total fare display which includes base fare, FED tax, airport charges, and service charges.
- Hospital and clinic finder in proximity using your current location.
- Weather alerts about destination cities in Pakistan.
- User-friendly interface for all kinds of users.

5.1.5 Customer Relationships:

- Self-service through the mobile application for booking and managing your profile.
- Personalized travel assistance through the AI assistant.
- In-app notifications regarding booking and booking confirmation.
- Email confirmations after each booking.
- In-app messaging with admin replies option.

5.1.6 Channels

- Primary channel: Android mobile application.
- Secondary channel: web browser support for users who do not have Android devices.
- Google Play Store for future release.
- Social media channels for marketing purposes and acquiring users.
- Word of mouth among travelers of Pakistan.

5.1.7 Cost Structure

The Travello AI is a cost-based product using free tiers and affordable services to reduce costs.

- Costs for using the API's of AviationStack, RapidAPI, and Google API.
- Free-tier hosting of backend on Render.com.
- Free-tier Supabase for database management and authentication.
- Time and labor costs of developing and maintaining the product from the project group.
- Use of Gmail SMTP for email delivery.

5.1.8 Revenue Streams

- Per booking charge for each completed booking through the website.
- Commission for each verified airport driver booking ride successfully.
- Featured listing of hotels and airlines for gaining visibility on the website.
- Advertising on the website by hotels, airlines, and tour operators.
- Subscription-based premium AI assistant features in the future version.

Chapter 6

System Development Methodology

6.1 Development Methodology

In the modern software solution, the development approach is an important strategic decision. Every methodology puts emphasis on risk management, involvement of stakeholders and timelines and flexibility to changing needs in different ways. In the Travello AI booking system, the use of Agile aligns with the evolving business objectives, stringent user experience expectations, and need for iterative, incremental delivery. This approach enables the team to maintain a steady but flexible development rate across the product's life, to take feedback from stakeholders and to regularly review assumptions.

Agile SDLC Phases for Our Project

Agile can be divided into project-specific phases even though it is not strictly linear:

- **Planning**

Focus on booking flights, hotels, trains, and cars with an integrated AI assistant to define the Travello AI product vision. Make the first backlog items, such as the booking confirmation, seat/room selection, search user interface, and results listing. Prioritize tasks and get ready for the first sprint.

- **Requirements Gathering**

Gather functional needs from stakeholders and consumers. Determine the demands of the business, such as quick search for flights, hotels, trains, and automobiles; safe card-based payment; a responsive user interface; and multi-platform compatibility for the Web, iOS, and Android. Record the non-functional requirements, such as linguistic support, performance, usability, and dependability.

- **Design**

Make UI flows and wireframes for user journeys and reservations. Create the system architecture for data storage, API integration, and mobile and web platforms. Describe visual design patterns, navigation flows, and reusable parts.

- **Development**

Implement features in brief sprints or iterations. Develop the search and booking core initially, then user accounts, and finally other services to build the system incrementally. Make use of shared code review and continuous integration.

- **Testing**

Run UI, integration and unit tests for each sprint. Check data flow, edge cases, form behavior and booking accuracy. To find flaws early, include quality tests following each increment.

- **Deployment / Release**

Provide access to working increments to stakeholders/environment. Once some of the modules are stable, deploy them to production starting with the main booking system up to the improvements. After every release, get input from stakeholders and users.

- **Maintenance and Iteration**

Examine sprint results and make improvements to the backlog. Prioritise based on new needs, analytics and feedback. Continue to make improvements in the next few Agile cycles.

JUSTIFICATION

Why Agile is the Best Choice for This Project

Agile is the most suitable approach for the Travello AI project due to the following reasons:

- **Requirement Changes Are Expected**

Travel applications constantly need to be updated for new regulations on booking, UI/UX improvements, and integrations.

- **User Feedback Matters**

We may swiftly deliver essential features and improve them in response to actual user behavior.

- **Incremental Delivery Speeds Value**

We can add more elaborate features in future sprints, once we have launched a minimum viable booking procedure.

- **Collaboration Is Essential**

Agile promotes great cooperation among stakeholders, developers, designers, and testers.

- **Quality Improves Over Time**

Regular testing and review cycles maintain the product in line with expectations and minimize problems.

In short, Agile gives a travel booking system the velocity, agility and control it requires, thanks to feedback. It slowly builds the product, and allows our team to adjust quickly to changes.

6.2 Key Milestone

Num.	Milestone	Outcome
1	Requirements & literature review	Comparative study of existing travel booking platforms; functional and non-functional requirements drafted
2	Frontend application development	Flutter app built across Android; booking, payment, profile and AI chat screens completed
3	Authentication & database setup	Supabase JWT authentication wired in; row-level security policies applied; core tables (profiles, bookings, payments) migrated
4	System & backend architecture design	Layered backend structure defined: routers, services, models, core; Supabase (PostgreSQL) schema planned
5	Flight booking module	Domestic route dataset built; AviationStack API integrated as live supplement for same-day flights
6	Hotel booking module	Multi-source hotel search implemented (primary provider with map-based fallback sources)
7	Train & car rental modules	Pakistan Railways dataset built for train search; car rental module added with driver assignment
8	Payment & booking flow	Card-only payment flow implemented; booking lifecycle (pending → confirmed → cancelled); e-ticket email via Gmail SMTP
9	AI assistant: language model integration	Groq set up as primary model provider with OpenRouter as fallback for quota limits
10	AI assistant: multi-agent system	Master orchestrating agent built, routing to specialist agents (booking, hotel, transport, weather, itinerary, etc.)
11	Integration testing & bug fixing	Automated test suites added covering booking recovery, multi-step bookings, and AI assistant edge cases
12	Final documentation & defence	Technical report, dissertation, and final presentation prepared

6.3 Project Plan with Gantt Chart:

No.	Project Activities	Jul'25	Aug'25	Sept'25	Oct'25	Nov'25	Dec'25	Jan'26	Feb'26	Mar'26	Apr'26	May'26	Jun'26	Jul'26	Aug'26
1	Requirements & literature review														
2	Frontend application development														
3	Authentication & database setup														
4	System & backend architecture design														
5	Flight booking module														
6	Hotel booking module														
7	Train & car rental modules														
8	Payment & booking flow														
	Mid Year Evaluation: 13 May'26														
9	AI assistant: language model integration														
10	AI assistant: multi-agent system														
11	Integration testing & bug fixing														
12	Final documentation														

Legend:  Process  Suggested Milestone

6.4 Tools Used for Development

A number of tools related to database administration, backend programming, and mobile app development were needed to complete this project. The tools that are really utilized and the minimal needs are listed below.

6.4.1 Hardware Requirements

To operate the mobile app emulator, the backend server, and the development tools simultaneously, a Windows personal computer with at least 8 GB of RAM and 50 GB of free hard drive space was needed. Because travel, hotel, train, weather, and AI functions all rely on real-time third-party services, a steady internet connection was also necessary during development.

6.4.2 Software Tools

Tool	Purpose
Flutter SDK	Building the mobile and web application from a single codebase
GetX	State management, dependency injection and named routing in the Flutter app
Python	Backend implementation language
FastAPI	Asynchronous REST API framework; Framework used to build the backend server and its API
Uvicorn	ASGI server hosting the FastAPI application
Supabase	Hosted database, used to store user, booking, and payment data, and to handle login
Visual Studio Code	Main code editor for both the app and backend
Android Studio	Running and testing the app on an Android emulator
Postman	Testing backend API endpoints during development
Swagger UI / ReDoc	Interactive API documentation at /docs and /redoc
Git & GitHub	Version control and storing the project source code
pytest	Automated unit, integration and adversarial testing of the backend
Render	Cloud platform hosting the deployed FastAPI backend
Figma	Designing the app's screens before building them
draw.io	Use case, sequence, activity and architecture diagrams
Lucid Chart	GUI diagram

Table 6.4: Software tools used in development

6.4.3 External Services and APIs

Service	Purpose in the System
RapidAPI Hotels	Primary hotel inventory source
Google Places API	Secondary hotel source when the primary provider returns no result
OpenStreetMap (Nominatim & Overpass)	Geocoding and tertiary hotel and location data
OpenWeatherMap	City weather data used by the weather module and the AI weather agent
Groq API	Primary large language model provider (llama-3.3-70b-versatile)
Open Router	Fallback language model provider (openai/gpt-oss-20b:free)
Email	Gmail SMTP for Transactional e-mail, booking confirmations and e-tickets

Table 6.5: External services and APIs

6.5 System Implementation

6.5.1 Implementation Overview

Three interconnected components make up Travello AI. The first is a mobile application that runs on an Android codebase and was created once in Flutter. The second is the backend server, which is constructed in Python using the FastAPI framework and houses all of the business functionality, including booking, payment processing, flight, hotel, and train searches, and the AI assistant. The third is Supabase, a PostgreSQL-based hosted database service that manages user login and contains all user, booking, and payment data. In order to keep API keys and passwords safe on the server rather than within the app itself, the application never communicates directly with any external service; instead, all requests pass through the backend.

6.5.2 Backend Structure

The backend is set up so that every component of the system has a single, distinct function. A routing layer receives incoming requests and verifies that the user is logged in and that the request is well-formed. Underneath it, a different service layer handles the actual job, such as finding a provider, figuring out a fare, or saving a reservation. This division makes it easier to maintain the system when features were added over the course of the two semesters. For instance, a change to the way flights are searched does not necessitate altering the code that manages logins or payments.

6.5.3 Authentication and Database

Supabase's authentication mechanism manages user accounts. Supabase provides a signed token that verifies a user's identity when they log in. The backend verifies this token on any request that involves personal information, so a request without a valid token is denied before it can access any booking or payment details. The database itself is set up as a second line of defense such that, even in the event of an error in the code, a user can only ever view or change their own entries. In order to prevent the user's entered information from being lost in the event that a payment fails midway through, bookings go through a straightforward lifecycle: pending when they are first generated, confirmed if payment is successful, or cancelled if the user cancels them.

6.5.4 Travel Modules

Each of the backend modules for the flights, hotels and trains were written to work even if an external data provider was slow, limited or temporarily unavailable—an important feature for a student project that relies on free-tier APIs. . In a predetermined order, each module reverts to a different source:

Module	Primary Source	If Unavailable
Flights	Curated real domestic routes and schedules	Supplemented with live flight-data API where the free plan allows
Hotels	Primary hotel data provider	Falls back to alternative map and location-based sources
Trains	Realistic dataset of actual Pakistan Railways routes and schedules	Not applicable; no public Pakistan Railways API exists

The curated route data for future-dated searches is supplemented rather than replaced by the flight-data API's free plan, which only supports same-day flights and a restricted number of monthly requests. There was no public API for Pakistan Railways so the project supervisor confirmed that the train module's use of a realistic dataset and not a live API is a good approach.

6.5.5 Booking and Payment

The app only takes card payments. When you make a booking and pay for it, the system behind the app says the booking is done. Then it uses Gmail's SMTP service to send you an email with all your travel plans and ticket information. If the email does not send it is okay. Your booking is still fine. The app makes sure your payment and confirmation are safe even if the email system is not working right. The booking itself does not change, no matter what happens with the email.

6.5.6 AI Assistant Implementation

The AI assistant is the part of the system that's different from the rest. This is because one single instruction to a language model is not enough on its own for something exact as a booking. Instead one coordinating agent reads the user's message. This agent figures out what the user is asking for. When needed the request is passed to one of special agents. These agents are responsible for tasks like finding hotels, comparing transport options or answering questions about the weather. Before any of these agents can make a booking there is a validation layer. This layer checks that the important details, like dates, cities and number of passengers were actually given by the user. It makes sure the model did not guess these details. Problems found during testing led directly to the creation of this validation layer. Each fix came with an automated test. This test ensures that the same mistake does not happen again..

6.5.7 Deployment

The backend is all. It is now on Render. Render is a place on the internet where we can put our website. This means our website is always running. It gets checked all the time to make sure it is working properly. If something goes wrong with the website we can fix it. Get it running again. The backend is on Render. It keeps running all the time.

Chapter 7

System Testing and Evaluation

7.1 Introduction

7.1.1 Purpose

This chapter is about how Travello AI was tested before it was ready to be looked at. It talks about how we made sure each part of the booking system, like flights, hotels, trains and cars works well on its own. We also checked that these parts work correctly together when someone is actually making a booking.. We made sure the Travello AI travel assistant, which is what users interact with, behaves safely and does what it is supposed to do even when it gets weird or wrong information. We did not test Travello AI with users, instead we tested it directly with the backend code, which is the part that makes everything work. This was all part of the project's automated tests that run by themselves.

Objectives

Three objectives guided the testing for this project:

- First of all, we aimed to provide customers with bookings as soon as they make them. Also, it was essential for us to prevent any unintentional errors from happening in the process of booking, as it is vital.
- The next step to perform was to test the AI assistant itself which communicates with customers and assists them in the process of making bookings. It should never be tricked into crashing or being frozen. Also, no booking information, such as date, city, vehicle or flight or its cost should be created by the AI assistant which was not asked about by the customer. That is why we had to make sure that the AI assistant creates bookings correctly.
- Another task for us was to ensure that we will be able to solve any issues we encounter during the testing and that it will not happen again. That is why, once we discovered some defects in the process of testing, we created some tests to prevent them from happening again. Thus, in case the code is changed by other developers in the future, we will know about it immediately and there will not be any recurrence of the defect again.

7.2 Test Methodology

The creators of Travello AI tested it using two testing approaches: manual testing and computer-based testing. The testing was carried out constantly during the development process rather than being done at the end of the process.

- Travello AI was tested as a whole product; the flights, hotels, trains, automobiles, payment systems and AI assistant were all tested. Each element was tested manually with the help of the program, and any problems found during testing were fixed.

- Computer-based testing was used in order to test the functionality of AI assistant, since it was difficult to test it manually. The assistant had to recognize user requests and generate answers to them, so it was necessary to ensure its correct operation. Five tests were conducted to check how Travello AI assistant would react to receiving weird and incorrect information from users. Such testing ensured that there was no crashing of the assistant and that it did not generate booking details based on the wrong user information.
- Flight, hotel and train sections were tested in order to ensure proper work of these sections together with companies' services. The test was carried out not only with test data but also with actual company services.

7.3 Test Plan

The testing phase was structured around the six functional requirements detailed in Chapter 3 (Section 3.2), which are provided here to ensure this chapter remains comprehensive as a standalone section.

ID	Requirement
REQ-1	User Authentication & Management: The system will let users sign up with an email and password. It will check that the email and password are correct. When users log in they can choose to stay logged in. If they forget their password they can get a code sent to their email to reset it. Users can change the information in their profile, change their password and even delete their account completely. This will get rid of all the information related to their account. When users are logged in they will stay logged in even if they close the app until they choose to log out.
REQ-2	Multi-Service Booking System: The system will help people book flights, trains and hotels. When people use the system they can look for services in a place and on a specific date. They can also use filters to find what they want. For example they can choose their seats on a flight or train. Pick a hotel room. The system will show them all the details they need to know. When they are ready they can finish booking their flight, train or hotel. Before they can do that they have to enter information about the people traveling like their names and stuff and the system will check to make sure everything is correct. This is important for flight bookings, train bookings and hotel bookings.
REQ-3	Payment Processing & Booking Management: The system will allow users to pay for purchases using credit cards and debit cards. These payment methods are authenticated by the use of Luhn validation technique. The system will also authenticate the use of CVV and expiry date of the cards. After making a booking, a user receives an e-mail that confirms the booking. The booking history is available for viewing. The user can view the status of the booking. Booking cancellations are also possible if needed. After a booking has been made, users are allowed to provide a rating along with some comments.
REQ-4	Search & Filtering System: It will have an intelligent search facility with auto complete options for cities, airports, and train stations. The user will be able to filter results based on price, time, number of stops, and

ID	Requirement
	even by choice of operator, and sort the results based on cheapest, fastest, and recommended.
REQ-5	User Support & Additional Features: The system shall provide a support messaging feature with query tracking and admin replies. Users receive in-app notifications for bookings and updates. Additional features include home city personalisation, healthcare search with maps and navigation, live weather updates for Pakistani cities, and onboarding screens for first-time users.
REQ-6	AI Travel Assistant: The system shall provide a conversational assistant that interprets travel requests in natural language and searches flights, trains, hotels and rides on the user's behalf. The assistant requests any missing trip details, presents a priced summary, and creates a booking only after the user explicitly confirms it. Card details and identity documents are never accepted through the chat and are collected only through the standard secure forms. Conversation history is stored so a user can resume an earlier chat.

7.4 Test Approach

Two testing approaches were used, matched to the nature of each requirement.

- The screens and flows that a user directly interacts with are covered by REQ-1 through REQ-5. Each of the app's features—register, search, book, and pay—was evaluated both manually and exploratorily, and the results were compared to the requirements. Instead of using a pre-written, independent test-case document, the developer used the program and treated any wrong behavior as a bug to identify its root cause.
- Since the AI assistant, REQ-6, uses its own decision logic to determine if a date, city, or vehicle in a booking was a user request or an invention, and if an incomplete request can safely be rejected (which can only be partially seen in the window), it needed a different approach. An automated, white-box regression suite was used to meet this need. The five test files in the backend/tests/ run directly the assistant validation and gating functions, in addition to humans testing the chat interface itself.

ID	Requirement	Test Approach
REQ-1	User Authentication & Management	Manual, exploratory (functional)
REQ-2	Multi-Service Booking System	Manual, exploratory (functional)
REQ-3	Payment Processing & Booking Management	Manual, exploratory (functional)
REQ-4	Search & Filtering System	Manual, exploratory (functional)
REQ-5	User Support & Additional Features	Manual, exploratory (functional)
REQ-6	AI Travel Assistant	Automated regression (white-box) + manual

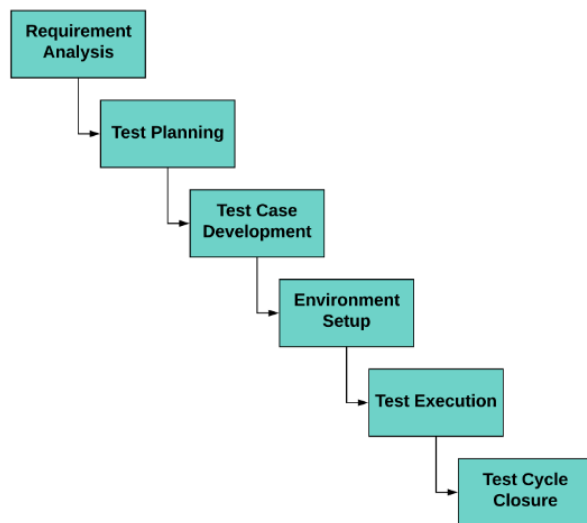


Figure 6.1: (TBD)

Requirement Analysis	The six functional requirements from Chapter 3 (REQ-1 to REQ-6)
Test Planning	Deciding manual vs. automated per requirement (Section 7.3/7.4)
Test Case Development	Formal only for REQ-6, the five automated suites are written test cases. For REQ-1 to REQ-5 this is scenario identification from the requirement, not a pre-written case document
Environment Setup	Local dev environment; mocked LLM and network layer for the automated suites
Test Execution	Manual use of the app, plus running the automated suites
Test Cycle Closure	Results consolidated in this chapter

7.5 Test Environment

Hardware

Component	Specification
Development machine	Windows 11 Pro, Intel Core i5, 8 GB RAM
Test device	Android smartphone, Android 11 or above, 4 GB RAM

Table 7.2: Test hardware environment

Software

Component	Version / Detail
Mobile framework	Flutter (Dart), Android release build
Backend framework	FastAPI (Python 3.11), served by Uvicorn
Deployment	Render
Database and authentication	Supabase (PostgreSQL)
AI service	Groq (Llama 3.3 70B), with as fallback OpenRouter
Email service	Gmail SMTP for ticket delivery
Version control	Git and GitHub

Table 7.3: Test software environment

7.6 Test Entrance Criteria

Testing of a given feature started after:

- Implementation of the corresponding functional requirement (Chapter 3) and building of the app free from compile or runtime errors.
- Setup of the test environment according to Section 7.5: the backend running on localhost, the Flutter app on an Android phone or emulator.
- Application of the Supabase database migrations, which ensured that the database tables with authentication, bookings, and other data were created and accessible.
- Configuration of the API keys needed for testing (Groq, Supabase, AviationStack, RapidAPI/Google Places, Gmail SMTP) so that the module could be tested in interaction with its actual dependencies rather than a partial implementation.
- As for the automated testing of the AI assistant in particular, the mocking of the language model and search services became necessary to enable execution of the five tests offline and getting consistent results.

7.7 Testing Acceptance Criteria

The automated regression suite must maintain a pass rate above 95%, all major functionality specified by the six functional criteria (Chapter 3) must function as intended, and no important bug—such as an inaccurate booking, a lost payment, or an AI assistant crash—may go unfixed. In actuality, the automated suite achieved a 100% pass rate (144 of 144 tests across all five test files), and all serious bugs discovered during manual testing were corrected and re-verified before being deemed closed.

7.8 Test Cases

Test Case ID	Test Objective	Test Data	Expected Result	Actual Result	Test Status	Bug ID
TC1	Log in with valid, registered credentials; session survives app restart	Valid email + correct password for an existing account	User authenticated, reaches home screen; session persists after restart unless logged out	As Expected	Pass	-
TC2	Recover password via email OTP	Registered email, "Forgot Password" requested, correct OTP entered before expiry	OTP is emailed; correct OTP allows setting a new password; login succeeds with it	As Expected	Pass	-
TC3	Search for and complete a domestic flight booking	Karachi to Islamabad, valid future date, 1 adult	Available flights are listed with fares and can be sorted and filtered; the selected flight is booked, status becomes paid and a PNR is generated	As Expected	Pass	Bg-6
TC4	Book a train and receive the ticket by email	Karachi to Sialkot, AC Standard, 1 passenger	Results within the filtered range returned; confirming guest details creates a booking	As expected after fixes	Pass	Bg-4, Bg-5, Bg-6
TC5	Search for and book a train seat	Lahore to Karachi, valid future date, 1 passenger	Available trains with seat availability returned; confirming details creates a booking	As Expected	Pass	-
TC6	Reject a card number that fails the Luhn check	Card number 4111 1111 1111 1112	Payment rejected before submission; no booking created	As Expected	Pass	-
TC7	Cancel an existing confirmed booking	A confirmed booking	Status updates to "cancelled" and reflects in booking history	As Expected	Pass	-
TC8	Auto-complete a city search and sort results by price	Partial city name typed; results sorted "cheapest"	Auto-complete suggests matching cities; sort reorders results by ascending price	As Expected	Pass	-

TC9	Submit a support query and receive an admin reply	A support message from a logged-in user	Query appears in support history with tracked status; admin reply visible and triggers a notification	As Expected	Pass	-
TC10	Assistant must not invent car-booking details from an incomplete request	Confirmed hotel booking, then "Now can you book car for me too?"	Assistant asks for missing drop-off, vehicle, and pickup time instead of booking with invented values	As expected after fixes	Pass	Bg-1
TC11	Hotel booking must not loop endlessly re-asking for a city already given	City given once ("book hotel in Multan"), listing picked, confirmed	Booking completes using the given city, without repeatedly re-asking	As expected after fixes Expected	Pass	Bg-2
TC12	A second car booking in one session must not inherit the first car's details	Car #1 fully specified (Sedan, 9am), then "book another car" with no details	Car #2 is not silently booked with car #1's data; assistant asks for car #2's own details	As expected after fixes	Pass	Bg-3
TC13	Assistant returns a well-formed response instead of crashing on hostile/malformed input	Oversized input, garbage tool arguments, prompt-extraction attempts, multi-domain messages	Every turn returns a non-empty, well-formed response; no unhandled exception	As expected	Pass	-
TC14	Complete a booking through the AI assistant	Chat: "Book me a flight from Lahore to Karachi tomorrow"	The assistant gathers the trip details, shows a priced summary, collects passenger details and creates the booking only after the user confirms	As expected after fixes	Pass	Bg-5, Bg-8
TC15	Save a booking with Pay Later, pay it from My Bookings and receive a notification	Any unpaid flight or train booking	The booking is stored with pending status and appears under My Bookings; payment can be completed from there, after which the status changes to paid and an in-app notification is generated	As expected after fix	Pass	Bg-4

7.9 Bug Reporting

7.9.1 Bug Tracking

For this project, no specific bug tracking software was utilized. Instead of being recorded in a different ticketing system like Jira or GitHub Issues, defects uncovered during manual testing were examined and addressed directly in the codebase. When this chapter refers to a defect by an ID (such as BG-01 in Section 7.8), that ID is not an identifier from an external tool, but rather a label used exclusively within this report to connect a test case to its fault explanation.

7.9.2 Bug Severity Definitions

Levels	Definitions
Critical	The problem causes reservations, wrong payments or charges a system crash or makes the AI assistant make a reservation, with information the person never gave. This stops any testing of that feature until it is fixed.
Medium	The problem makes the experience worse. Makes the user have to do the same thing over again (for example getting asked the same question many times) but it doesn't lead to a wrong reservation, a wrong payment or a system crash.
Low	A cosmetic, minor usability, or convenience issue with negligible impact on the application's core functionality.

Table 7.5: Bug severity definitions

7.9.3 Bug Priority Definitions

Levels	Definitions
High	Must be fixed before the current build is considered complete; typically a defect that blocks a core booking or payment flow, or causes the AI assistant to fabricate a booking detail.
Medium	Should be fixed, but does not block the current build; addressed in the next development pass.
Low	Fixed only if time permits before final submission; does not affect core functionality.

Table 7.6: Bug priority definitions

7.9.4 Bug Life Cycle

The bug life cycle has the following steps or status as shown in the diagram..

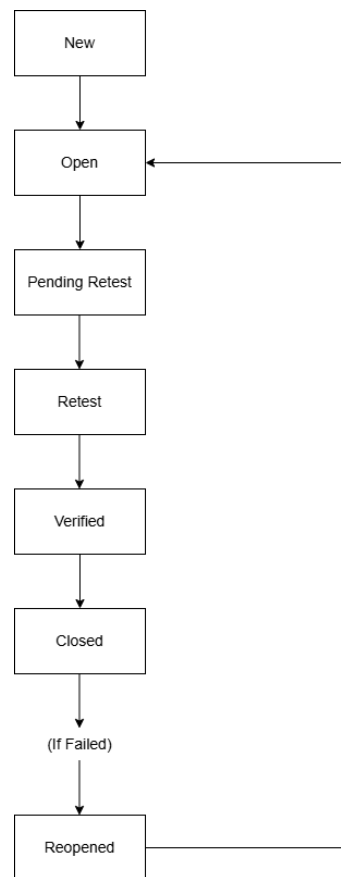


Figure 7.2: Bug Life Cycle

- **New:** A problem is found when people test manually or when an automated check fails in Travello AI.
- **Open:** The problem is. People are trying to find out why it happened in the Travello AI application or in the AI logic.
- **Fixed:** A change is made to the code or the AI logic to fix the reason for the problem.
- **Pending Retest:** The fix is done and waiting to be checked
- **Retest:** The steps that first showed the problem or the related automated test are run again.
- **Verified:** The test again shows that the problem is gone.
- **Closed:** The problem is considered fixed. For problems in the Travello AI assistant a permanent test is also added so the problem cannot come back without being noticed.
- **Reopened:** If the test shows the fix was not complete the problem goes back to being closed.

7.9.5 Bug Reports

Bug ID	Test Case ID	Bug Description	Reported By	Status	Fixed by	Severity	Priority
Bg-1	TC10	The AI-assistant created a detailed and verifiable booking of a car, creating the place to be dropped off, the car type, and the time to pick up.	Syeda Tooba Fatima	Closed	Sameed Uddin	Critical	High
Bg-2	TC11	In the case of booking the hotel, when the user entered the destination city only once, the system kept on asking "Which city?" about six times before confirming the booking since the search tool had stored the field as a city, whereas the booking gate needed the destination.	Huzaifa Rizwan	Closed	Sameed Uddin	Medium	High
Bg-3	TC12	Since the scope of logic began with the mention of the first car and not the last one, the type of vehicle and the pickup time automatically applied to the second car booking without raising any warning messages.	Syeda Tooba Fatima	Closed	Sameed Uddin	Critical	High
Bg-4	TC4, TC15	Closing the card-payment sheet without paying silently discarded the pending booking instead of leaving it saved as pending under My Bookings.	Huzaifa Rizwan	Closed	Sameed Uddin	Critical	High
Bg-5	TC4, TC14	The emailed ticket showed the passenger's name prefixed with "None" when the title field was left empty.	Syeda Tooba Fatima	Closed	Sameed Uddin	Low	Medium
Bg-6	TC3, TC4	Booking references were generated using UTC time, so a booking made late at night could be stamped with the previous day's date.	Syeda Tooba Fatima	Closed	Sameed Uddin	Medium	Medium
Bg-8	TC14	Bookings for multiple passengers arranged through the AI assistant were charged per seat rather than per party, hence overcharging a booking by the number of passengers.	Huzaifa Rizwan	Closed	Sameed Uddin	Critical	High

Table 7.7: Bug reports

7.10 User Acceptance Testing

User# 1 Review :			
Name		Signature	

User# 2 Review :			
Name		Signature	

User# 3 Review :			
Name		Signature	

Chapter 8

Result and Discussion

8.1 System Architecture and Component Communication:

The system is made up of a client and a server. The client, which is an application for Android made with Flutter, does not connect to the database directly. Instead it uses HTTPS to talk to the FastAPI backend. The backend does two things at the same time. It talks to the user in a conversation like when the user's speaking to the AI assistant and it also uses traditional REST endpoints like when the user is searching or booking something. The AI assistant uses a language model called Groqs hosted llama-3.3-70b-model. If this model is not available the system uses OpenRouter models instead. The service layer is where all the data is stored, in a database called Supabase, which's like PostgreSQL with a special kind of security. The system also sends emails using Gmail. Gets information from other services like AviationStack for flights and Google Weather for the weather.

The system works the way whether the user is talking to the AI assistant or navigating through the application. For example if the user makes a reservation by talking to the AI assistant or by clicking through the application the system checks the reservation, in the way and stores it in the same database. The system uses the service layer for everything, which makes it simple and easy to manage. The Flutter mobile application only talks to the FastAPI backend using HTTPS and JSON which makes it safe and secure.

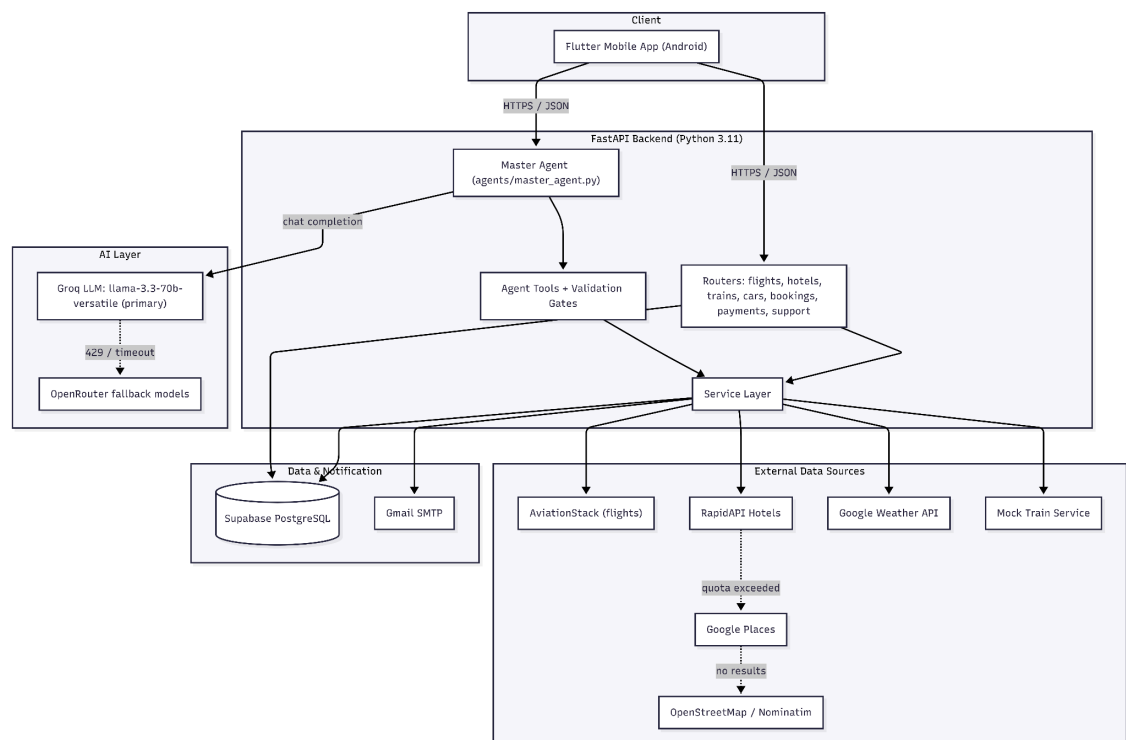


Figure 8.1: High-level system architecture

8.2 Request-Response Flow: AI-Assisted Booking

This part is about a person who wants to book a car. We will follow this person's request from start to finish. The person uses the Flutter app to send a message to the people in the back who then send it to the master agent. The master agent is the one who gets the message from the person who wants to book a car. The person uses the Flutter app to ask for help with booking a car. The people in the back get the message. They send it to the master agent. Along with a list of the tools that are available, the agent transmits the conversation to the LLM (search flights, book hotel, book car, etc.). The call is not immediately carried out if the LLM chooses to contact `book_car` with a particular pickup location, car, and time. It initially goes through a validation gate, which verifies that each of those values is present in the user's own words from earlier in the conversation. The reservation moves via the service layer into Supabase and a confirmation is sent back if everything traces back appropriately. Instead of silently completing a booking with false information, the gate blocks the call and the agent asks the user for the missing detail if the model has created a value (a common LLM failure mode).

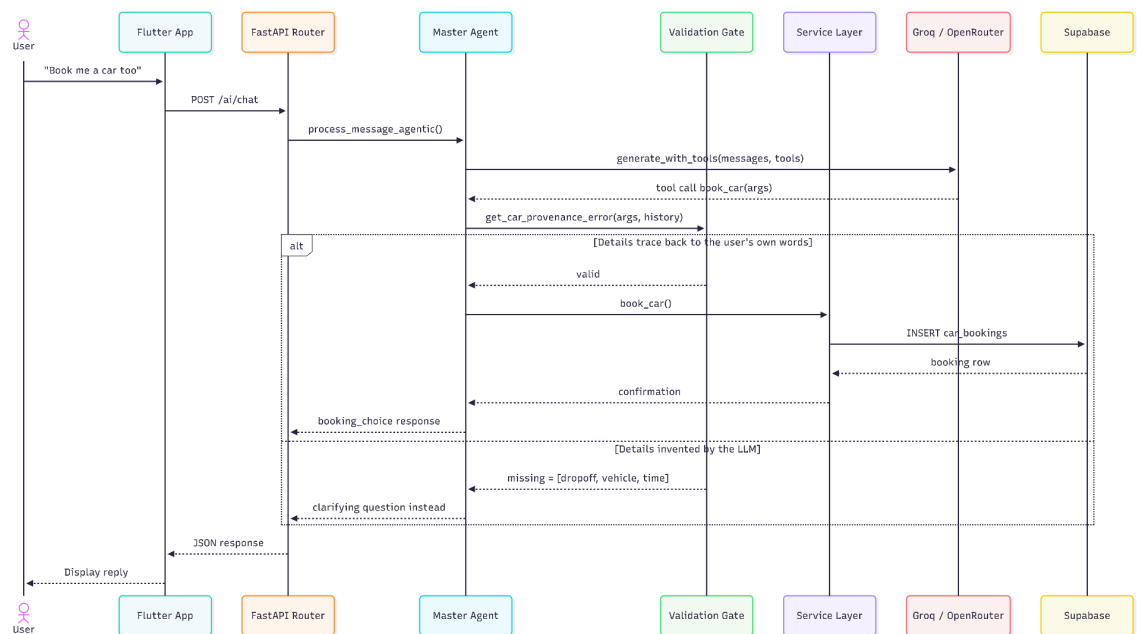


Figure 8.2: Request-response flow for an AI-assisted booking

8.3 Backend Validation Logic:

Even when they are incorrect, large language models provide output that sounds plausible and fluid; this failure pattern is sometimes referred to as hallucination. It is not okay to believe the model's answer without checking because a trip booking assistant starts actions (sending a driver creating a booking that can be paid for). So, in this system every tool call that is important for bookings goes through a gate made with simple Python code instead of another model response. In order to prevent, for instance, an earlier hotel check-in date from silently grounding a later, unrelated car booking's pickup time, the gate does not assess whether a value appears reasonable; rather, it determines whether the value can be traced to something the user actually said, scoped to the appropriate part of the conversation.

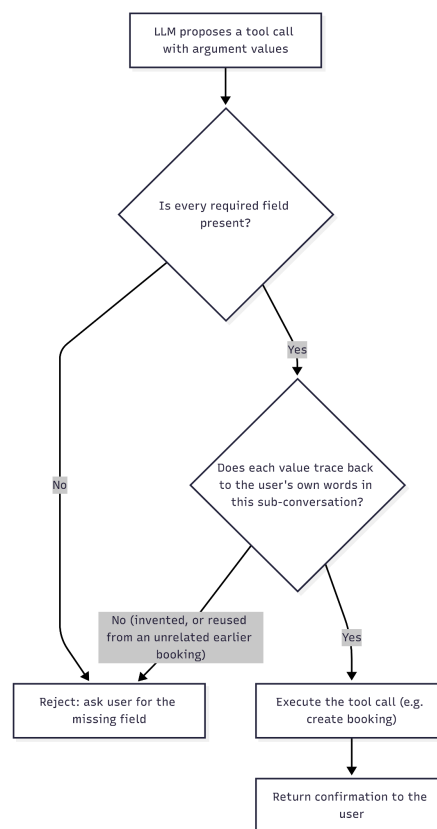


Figure 8.3: Deterministic validation gate logic

8.4 External Service Integration and Fallback Behavior

The system is dependent on a number of third-party services that can and do malfunction in production: timeouts, monthly quota exhaustion, and rate limitations are typical, not unusual. Every domain has a fallback chain built in so that if one provider fails, the user's request is still fulfilled.

Groq is the main model for the AI layer since it is quick and free-tier; if it returns a rate-limit error or is in a cooldown period, the system automatically retries against a list of OpenRouter models sequentially, omitting any that also return a 429. RapidAPI Hotels is the main source for hotel searches; if that doesn't work, Google Places, OpenStreetMap/Nominatim, and Lastly, a collection of seeded mock listings that combine any accessible real results. Flight search is broken down by route: international routes use AviationStack as the primary source with a dummy supplement, while domestic Pakistani routes use seeded data supplemented by AviationStack (limited to same-day real-time flights on its free tier). The main API we use for weather data is the Google weather API. If the Google weather API is not available. If it is slow because too many people are using it, our system will automatically start using the Open-Meteo API so we can still show the weather forecast. We do not have an API for Pakistan Railways so we have to make up the train data. Our project supervisor said this is okay. It is a good way to do things for now rather than trying to connect to the Pakistan Railways system, which is not possible.

When we were checking everything out we saw that the system did what it was supposed to do. It had a problem with a hotel search. The system got a message from RapidAPI that said it had used up all its requests. So the system switched to Google Places. This worked well because Google Places was able to give us 20 real hotel listings for the same search. The best part is that the person using the system did not even notice that anything had gone wrong with the request from RapidAPI. The hotel search request from the user still worked fine because the system had switched to Google Places to get the hotel listings. This is how the system was designed to work with the RapidAPI hotel search and Google Places.

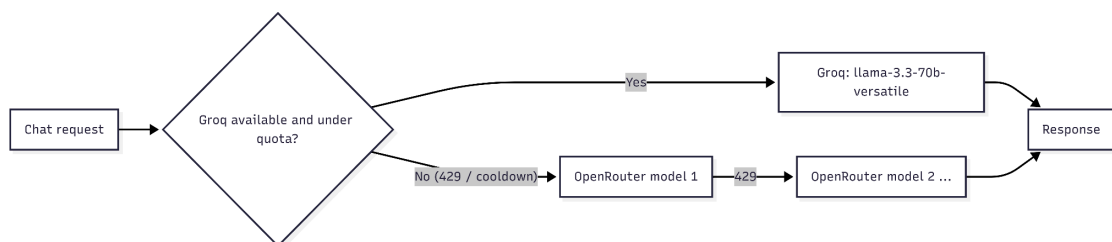


Figure 8.4: LLM fallback chain

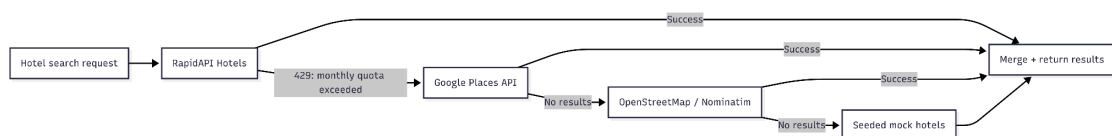


Figure 8.5: Hotel search fallback chain

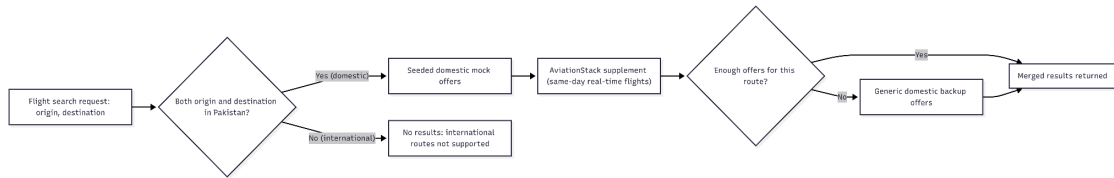


Figure 8.6: Flight search logic

8.5 Results: Functional Correctness (Automated Regression Testing)

An accuracy measure specific to classification does not apply here, as the AI assistant is built using a language model that is hosted elsewhere and was not trained as part of this project; otherwise, it could have been included and may have been misleading. What is measurable and has been measured is the behavior of the system and specifically whether the gates described in section 8.4 are properly detecting the failure scenarios that they were created to detect. To determine the deterministic results, nine regression tests have been performed.

8.6 Results: API and AI Response Performance

The backend was run on a machine using its real settings (a real Supabase connection, real API credentials). A small number of requests were sent to each endpoint. Response times were captured directly from the server to check how the system actually behaved by making guesses. Only read and search endpoints and a direct LLM connection were used for this test. No reservations or fees were involved.

Operation	Observed latency	Notes
Health check	5 to 57 ms	Baseline server overhead
Flight search (domestic & international)	1 to 10 ms	Served from seeded/mock data, no external call on this route
Train search	1 to 20 ms	Fully mock service
Hotel search	1.86 to 2.66 s	Included a real RapidAPI quota failure and live fallback to Google Places (Section 8.5)
AI assistant reply (direct LLM call, ~300-token response)	0.80 to 2.90 s	Real call to Groq's hosted model

These should be seen as ideas about how fast things might be instead of strict numbers because they come from three tests for each action. The times under 20 milliseconds for flights and trains show that routes using pre-loaded data respond very quickly no matter what is happening with outside services. The wider range, for hotel searches and AI answers shows network and third-party processing times instead of slow backend systems.

Chapter 9

Conclusion and Future Work

8.1 Conclusion

[Include a brief summary of how the proposed solution is going to/has addressed the problem statement specified in the introduction section. Provide an overview of what kind of evaluations were undertaken in order to prove that the solution really solves the problem with evidence on results findings.]

8.2 Future Work

[Provide an overview of the recommendations and include future directions which is required as part of the future work.]

References:

- [1] Booking.com, "Official Website," [Online]. Available: <https://www.booking.com>. [Accessed: May 12, 2026].
- [2] Sastaticket.pk, "Official Website," [Online]. Available: <https://www.sastaticket.pk>. [Accessed: May 12, 2026].
- [3] Skyscanner, "Official Website," [Online]. Available: <https://www.skyscanner.com>. [Accessed: May 12, 2026].
- [4] Bookme.pk, "Official Website," [Online]. Available: <https://www.bookme.pk>. [Accessed: May 12, 2026].
- [5] K. S. Reddy, N. Sruthi, N. V. Chand, and N. N. Reddy, "AI-Powered Personalized Travel Recommendation System," *International Journal of Engineering Research and Science & Technology*, vol. 22, no. 1, pp. 387–398, 2026.
- [6] A. Prashant, A. M. Doddamani, A. P. Kurian, and A. Sutradhar, "Pocket Safar: An AI-Driven Smart Travel Recommendation and Dynamic Itinerary Planning System," *International Journal for Research in Applied Science and Engineering Technology*, 2025.
- [7] H. Chitaliya, D. Jain, S. Tiwari, G. S. Khatri, D. Sankhe, P. Kanani, and M. Bhanja, "ExplainableTrip: An XAI-Driven Personalized Travel Recommendation System Using Gradient Boosting and SHAP," *Ingénierie des Systèmes d'Information*, vol. 30, no. 5, pp. 1189–1199, 2025.

Appendices

Appendix 1: User Manual

Appendix 2: Wall Poster

Appendix 3: Boucher

Appendix 4: Standee

Appendix 5: Research Paper / Publication / Publication acceptance letter

PAGE FOR REMARKS / COMMENTS

PAGE FOR REMARKS / COMMENTS